

Max

A private, local voice assistant

Technical guide: how it works and why it is built this way

Covers: Phases 1-4 (voice loop and laptop control; web research and browser control; memory, reminders and the dashboard; the Android app and remote access) plus Canvas, the daily digest, the desktop overlay and meeting/lecture notes with live captions.

Status: October 1, 2026. 102 automated tests passing.

Code: github.com/ManasKhare3005/Hey-Max

Contents

1. What Max is	4
1.1 Constraints that shape every decision	4
1.2 Build phases	4
2. Architecture at a glance	5
2.1 One command, step by step	5
2.2 Where things run	6
3. The voice pipeline	6
3.1 Microphone input (audio.py)	6
3.2 Wake word (wakeword.py)	6
3.3 Listening and end of turn (listen.py)	7
3.4 Speech-to-text (stt.py)	8
3.5 Voice output and conversation (tts.py, main.py)	8
4. The agent (agent.py)	8
4.1 Two models, one escalation tool	9
4.2 Direct replies	9
4.3 Prompt design for speed	9
4.4 History that keeps tool calls	9
4.5 The safety gate	9
4.6 Memory in every request	9
4.7 Guarding against claimed actions	10
5. Running the language model (llm.py)	10
5.1 The speed investigation	11
6. Tools	11
6.1 The tool registry (tools/registry.py)	11
6.2 Laptop tools (tools/system.py)	11
6.3 Web search and research (web.py)	12
6.4 Max's own browser (browser.py, tools/browser.py)	12
7. Memory and reminders	13
7.1 The memory store (memory.py)	13
7.2 Reminders (reminders.py)	14
8. Canvas and the daily digest	14
8.1 Canvas through the calendar feed (canvas.py)	14
8.2 The daily digest (digest.py)	14
9. Meeting and lecture notes (notes.py)	14
9.1 Live captions	15
9.2 Study summaries	15
10. Dashboard, API and desktop overlay	16
10.1 Event bus and API (events.py, server.py)	16
10.2 The dashboard (dashboard/)	16
10.3 Always on: tray, auto-start and overlay	16
11. The phone app and remote access (Phase 4)	17
11.1 Reaching the laptop from anywhere	17
11.2 What the app does	17
11.3 Approvals from the phone	18
11.4 Appearance: colours, centrepieces and fonts	18
11.5 Pages open on the phone	19
11.6 Max acts on the phone	19
12. Reliability on Windows	20
12.1 The "only listens when its window is in front" bug	20
12.2 Heartbeat and stack dumps (diagnostics.py)	20
12.3 Launching	21
13. Configuration reference (config.yaml)	21

14. Project layout.....22

15. How it is tested23

16. Key lessons.....23

17. What comes next.....24

1. What Max is

Max is a voice assistant that runs entirely on a Windows laptop. You say **"Hey Max"**, give a command, and it answers out loud: opening apps, controlling media, searching the web, and operating its own Chrome window. The wake word, speech recognition, the language model and the voice all run locally; nothing is sent to a cloud AI service.

The project has two goals of equal weight: a tool used every day, and a portfolio piece that shows clean architecture and strong agent design.

1.1 Constraints that shape every decision

Constraint	What it means in practice
Fully local LLM	Models run in Ollama on the laptop. No OpenAI/Anthropic APIs.
4 GB GPU (RTX 3050 Laptop)	About 3 GB usable after Windows. Only one small model fits on the GPU at a time, so everything else (wake word, speech-to-text, voice) runs on the CPU.
16 GB RAM	Enough for the models on the CPU side plus Chrome.
Laptop is the host	The "brain" is a separate service so it can move to another machine later with a config change.
Safety first	Reads and harmless actions run automatically. Anything that sends, deletes, buys, closes apps or changes power needs an explicit spoken "yes"; silence counts as no.

1.2 Build phases

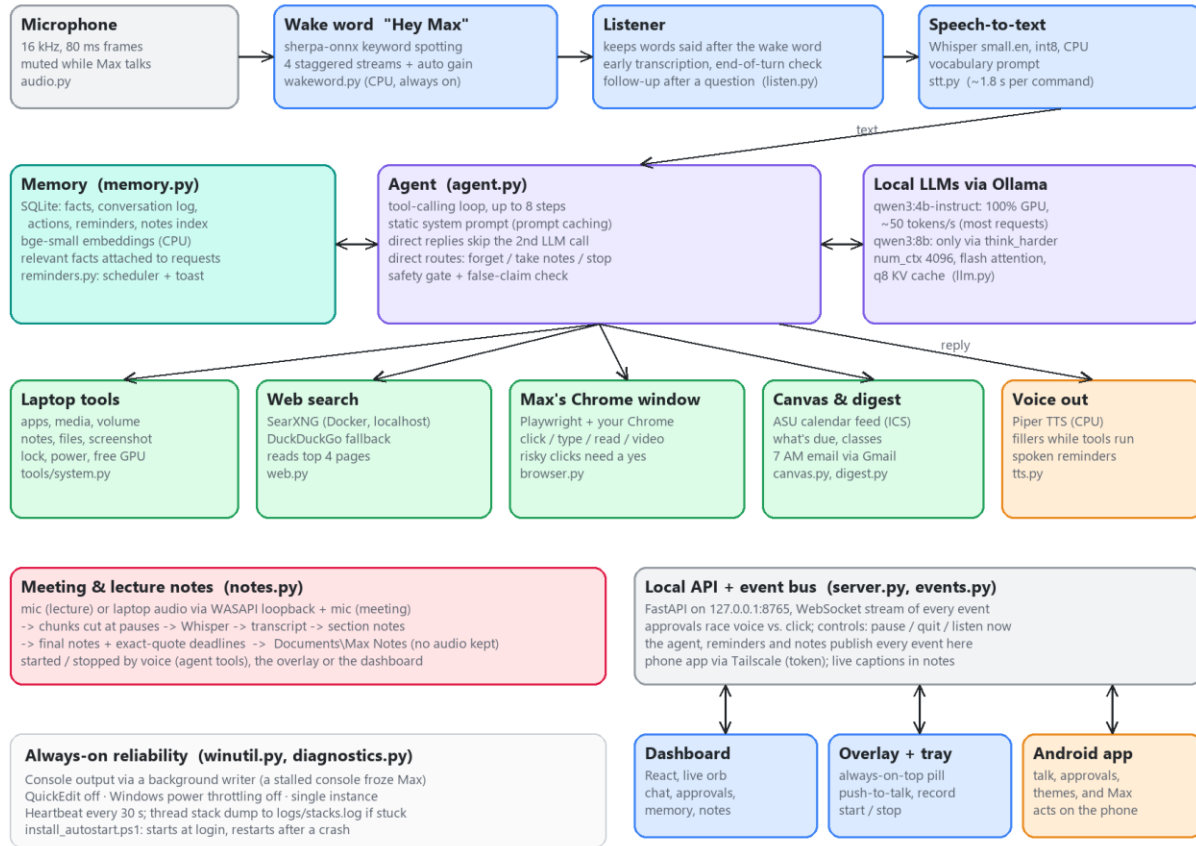
Phase	Scope	Status
1	Voice loop and laptop control	Done
2	Agent upgrade, web research (SearXNG), browser control (Playwright)	Done
3	Memory (SQLite + local vector search), reminders, React dashboard	Done
4	Android app, Tailscale, WebSocket link	Done
5	Galaxy Watch (Wear OS) companion	Next
6	Gmail / Calendar / Tasks, polish, demo video	Planned

Added along the way: Canvas integration and a daily digest email, background mode with a tray icon and auto-start, a desktop overlay, and meeting/lecture notes with live captions and study summaries.

2. Architecture at a glance

A command flows left to right and top to bottom: the microphone feeds the always-on wake word detector; after "Hey Max" the listener records the command, speech-to-text turns it into text, and the agent decides what to do, calling tools and the local language model, before the reply is spoken.

Max: architecture (everything runs on the laptop; no cloud AI)



Private settings (Canvas feed link, Gmail app password) live in secrets.yaml, which is never committed.

2.1 One command, step by step

1. **Wake word.** Every 80 ms of audio is fed to four keyword spotters. When one hears "Hey Max", a chime plays.
2. **Listening.** The listener keeps the audio that follows (so "Hey Max, open YouTube" works in one breath) and records until you pause.
3. **Early transcription.** After 0.4 s of silence, Whisper starts transcribing in the background. If the text looks like a finished sentence, the turn ends immediately; if it trails off ("search for..."), Max waits up to 1.8 s for the rest.
4. **Agent.** The text goes to the fast model (qwen3:4b-instruct) along with the tool definitions. The model either answers or calls tools, possibly several rounds.
5. **Tools.** Laptop actions, web search or browser actions run. Risky ones ask for a spoken yes first.

6. **Reply.** Tools that already return a sentence ("Opened Spotify.") are spoken directly; otherwise the model phrases the result. Piper speaks it.
7. **Follow-up.** If the reply is a question, Max listens again for 5 s without needing the wake word.

2.2 Where things run

Component	Technology	Runs on	Typical time
Wake word	sherpa-onnx keyword spotting (3.3M-parameter zipformer)	CPU, always on	~13% of one core
Speech-to-text	faster-whisper, small.en, int8	CPU	~1.8 s per command
Language model	Ollama: qwen3:4b-instruct (fast), qwen3:8b (planner)	GPU (4b, 100%)	0.4-2 s per turn
Voice	Piper (en_US-ryan-medium), Windows SAPI fallback	CPU	~0.3 s to synthesize
Web search	SearXNG in Docker + page reading	CPU / network	5-7 s with page reading
Browser	Playwright driving installed Chrome	CPU	~1 s per action
Memory search	fastembed bge-small-en (ONNX)	CPU	~28 ms per lookup
Dashboard & API	FastAPI + WebSocket; React/Vite UI	CPU, localhost	live
Meeting notes	Whisper in chunks + LLM section notes	CPU + GPU	~1 min after stopping

3. The voice pipeline

3.1 Microphone input (audio.py)

A `sounddevice` input stream delivers 16 kHz mono audio in 80 ms frames (1,280 samples) into a queue. While Max is speaking, the microphone is muted (frames are dropped) so it does not hear itself; afterwards the queue is flushed.

The microphone object also counts every frame it receives and the average input level. Those counters feed the heartbeat described in section 11.

3.2 Wake word (wakeword.py)

The original plan used openWakeWord's built-in "Hey Jarvis" model. After the rename to Max, a custom "Hey Max" model would have had to be trained, and openWakeWord's official training notebook turned out to be broken on current Google Colab (an open upstream issue). So Max switched to **sherpa-onnx**

open-vocabulary keyword spotting: the wake phrase is plain text in the config, and no training is needed.

Two tricks that make it reliable

- **Staggered streams.** The model decodes audio in fixed 0.64 s chunks and misses the phrase when it straddles a chunk boundary badly. Measured: a single stream missed about 1 call in 5. Max runs **4 streams started 0.16 s apart**, so one of them always sees the phrase well aligned. Any stream can fire.
- **Automatic gain.** Quiet audio is boosted up to 16x before decoding; loud audio is never turned down. The gain follows the loudest sample of the last 1.5 s, so a sudden loud word is not distorted.

Test (synthetic speech)	1 stream, no gain	4 streams + auto gain
3 calls, 1 s apart	16 / 24	24 / 24
Normal voice	21 / 24	24 / 24
Low voice (-30 dB)	15 / 24	23 / 24
False triggers (56 near-miss phrases + 60 s noise)	0	0

Settings were later retuned for quiet speech (threshold 0.12, boost 2, gain 16), again with zero false triggers. One known limit: "Hey Mac" sounds almost identical and also triggers. And once your voice is barely louder than the room noise, no setting helps, because gain boosts the noise too.

If the sherpa model cannot load, Max falls back to openWakeWord's "Hey Jarvis" so it still works.

3.3 Listening and end of turn (listen.py)

The listener decides when you have finished speaking. The simple approach (record until 1 s of silence, then transcribe) wasted time twice: waiting out the silence, then waiting for transcription. Max overlaps them:

- Audio right after the wake word is kept: the chime plays in the background and the microphone is muted only for its 0.2 s, instead of discarding everything while the chime played.
- A 0.6 s lead-in is kept before speech is detected, so quiet first words are not clipped.
- At 0.4 s of silence, transcription starts in a background thread. If you start talking again, that result is discarded.
- If the early transcript looks complete (it does not end in "..." or a word like "for", "the", "and"), Max answers straight away.
- If it still sounds unfinished at 1.8 s of silence, everything is transcribed once more, because a softly spoken last word can pass for silence.

- The tail of "Hey Max" sometimes lands in the recording; it is stripped from the start of the transcript.

3.4 Speech-to-text (stt.py)

Whisper small.en (int8, CPU, greedy decoding) is used for every command, with a short **vocabulary prompt** ("Voice commands to an assistant called Max: open YouTube, search for, close this tab...") that measurably helps with commands and app names.

Engine (test commands)	Clear speech: errors / time	Quiet + noisy: errors / time
Whisper small.en	2.1% / 1.8 s	10.8% / 1.8 s
Whisper base.en	3.9% / 0.6 s	18.8% / 0.65 s
Whisper distil-small.en	6.8% / 1.6 s	14.4% / 1.6 s
Moonshine base (sherpa-onnx)	2.1% / 0.2 s	38% / 0.2 s

Why the fast engine is off

Moonshine is 9x faster and matched Whisper on clear synthetic speech, so it was briefly used for clear recordings. On Manas's real voice it misheard badly ("close this tab" became "close the staircase", "search" became "third"). His microphone driver also silences the background to almost exactly zero, which made every recording look perfectly clean, so the routing always chose Moonshine. Whisper is now the default; Moonshine remains an opt-in setting.

3.5 Voice output and conversation (tts.py, main.py)

- **Piper** synthesizes speech on the CPU in about 0.3 s. If the voice file is missing, Windows' built-in voice is used.
- **Fillers:** when a slow tool starts (web search, reading a page, handing off to the big model), Max says a short line like "Let me look that up." from a background thread. A speaking lock makes sure it never talks over its own answer.
- **Follow-up listening:** if Max's reply ends in a question mark, it listens again for 5 s without the wake word. It waits 0.3 s first so the echo of its own voice is not recorded, and ignores phrases speech-to-text tends to invent from near-silence ("Thank you.", "you").

4. The agent (agent.py)

The agent is a tool-calling loop. Each turn it sends the conversation and the tool definitions to the model; the model either answers in text (done) or asks for tool calls, whose results are added to the conversation for the next round, up to 8 rounds.

4.1 Two models, one escalation tool

The fast model, **qwen3:4b-instruct**, handles almost everything. For tasks needing real planning or long writing it can call a special **think_harder** tool, which switches the rest of the turn to **qwen3:8b** (slower, partly on the CPU).

4.2 Direct replies

Many tools already return a finished spoken sentence: "Opened Spotify.", "Volume set to about 30 percent." Those tools are marked **direct=True**, and when every tool call in a step is direct, their results are spoken without a second model call. That halves the time for most commands.

- Not used for requests containing "and"/"then" (they may need more steps) or when a tool reports an error.
- A tool can still hand a result to the model by returning `ForLLM(...)`, for example a list of page elements to choose from.
- A declined risky action is answered with "Okay, I won't." directly.

4.3 Prompt design for speed

Ollama reuses the already-processed start of a prompt between requests. The tool definitions are inserted right after the system prompt, so the system prompt must stay identical from request to request. The current time therefore goes at the start of each user message ("[Tue Sep 29 2026, 4:13 PM] open youtube") instead of in the system prompt. Measured: about 1,600 prompt tokens reused per request, prompt processing about 0.05 s.

4.4 History that keeps tool calls

Early on, history stored only the text of each turn. After a few turns the conversation looked like "user asks, Max just answers", and the model copied that pattern: it claimed "I went back" or "Scrolled down" without calling any tool. History now keeps each turn's tool calls and results (results trimmed to 240 characters), within a 4,000-character budget so the context never overflows. An overflowing context would make Ollama cut the start of the prompt, which is the system instructions.

4.5 The safety gate

Tools declare whether they are risky (**risky=True** plus a confirmation sentence). Before a risky tool runs, Max asks the question aloud and listens; only a clear yes counts ("yes, go ahead" yes; "yes... no wait, cancel" no; silence no). Browser actions add dynamic checks, described in section 6.

4.6 Memory in every request

Before each request, the facts that match it best (by meaning, see section 7) are appended to the user's message as "(You remember: ...)". They go in the user message, not the system prompt, so the cached

prompt prefix keeps working. Only facts scoring above 0.68 are attached: in measurements, a question and the fact it is about scored 0.74-0.88, unrelated pairs at most 0.61.

4.7 Guarding against claimed actions

The 4b model sometimes said it had done something without calling the tool: "I've forgotten that", "I'll cancel your reminder", "I've stopped recording". Nothing had actually happened. Two defences:

- **False-claim check:** if a reply claims a remember/forget/reminder action (or the user clearly asked for one) and the matching tool wasn't called in that turn, the agent sends the model one correction ("you said that, but nothing happened; call the tool now").
- **Direct routes:** phrasings the model kept getting wrong go straight to their tool in code: "forget ..." (which always asks for confirmation), "take notes" (lecture or meeting, picked from words like Zoom or call) and "stop taking notes".

Result

Memory commands went from 3 of 6 correct to 8 of 8, and 15 of 15 notes phrasings route correctly, while quick notes like "note that I need milk" still go to the model.

5. Running the language model (llm.py)

Max talks to Ollama over its local HTTP API (`/api/chat` with tools). Key settings live in `config.yaml` under `llm`.

Setting	Value	Why
fast_model	qwen3:4b-instruct	The plain qwen3:4b tag now points to a "Thinking" build that always reasons: 300-2,000 hidden tokens per reply.
think	false	No reasoning mode (the 8b model supports switching it off).
num_ctx	4096	Fixed context size so VRAM use is predictable.
num_predict	256	Caps runaway replies.
fast_model_gpu_layers	99	Forces all layers onto the GPU; Ollama's own estimate left 1 GB unused. Falls back automatically if loading fails.
keep_alive	15m	Model stays loaded between commands.
OLLAMA_FLASH_ATTENTION=1, OLLAMA_KV_CACHE_TYPE=q8_0	(environment)	Halves the conversation cache so the model fits in VRAM. setup.ps1 sets them.

5.1 The speed investigation

The same five test commands were timed after each change:

Stage	5 turns	Per command
Start (qwen3:4b Thinking build, 1/3 on CPU)	346 s	30-135 s
Switch to qwen3:4b-instruct	14.0 s	2-4 s
Free leaked VRAM (orphaned model process)	8.4 s	1-2 s
All layers on GPU + direct replies + prompt reuse	4.4 s	0.4-1.9 s

The biggest lesson

The first reported bug, Max reading its own reasoning aloud, and the extreme slowness had the same cause: Ollama's qwen3:4b tag had become a model that cannot stop reasoning. Measuring token counts per reply (not just wall time) revealed it.

6. Tools

6.1 The tool registry (tools/registry.py)

Tools are plain Python functions registered with a decorator that describes them to the model:

```
@reg.tool("Open or launch an application...", params={"name": {...}}, direct=True)
```

- `params` becomes the JSON schema the model sees; `required` defaults to all of them.
- `risky=True` + `confirm="..."` puts the tool behind the spoken yes/no gate.
- `direct=True` means the result is a finished spoken sentence.
- A `ctx` parameter is injected automatically: shared state such as the config, the LLM client, the web searcher and the browser session.

6.2 Laptop tools (tools/system.py)

Tool	What it does
open_app / close_app	Launch by config alias or fuzzy Start Menu match; close is risky
open_website	Open a site, Google search (tabs: all/images/videos/news/shopping/maps), or search inside YouTube, Amazon, Wikipedia, Reddit, GitHub, Spotify, eBay
media_control, volume	Media keys (Spotify etc.) and system volume
take_note, read_notes	Notes file in data/
find_files, open_file	Search Desktop/Documents/Downloads/...; open by number

Tool	What it does
system_status	Battery, CPU, memory
screenshot, lock_screen	Saved to Pictures/Max; lock Windows
power, cancel_shutdown	Shutdown/restart/sleep (risky, 10 s delay, cancellable)
free_gpu	"Go to sleep": unloads the models before gaming
get_datetime	Current date and time

6.3 Web search and research (web.py)

Search runs on **SearXNG**, a private meta-search engine that queries Google, Brave and others, running in Docker on [127.0.0.1:8888](#). If Docker is not running, Max falls back to DuckDuckGo.

Search result snippets alone often lack the answer (for example a "Super Bowl winners by year" page), and the 4b model then filled the gap from outdated memory, even inventing a source. So [web_search](#) also **reads the top 4 pages** in parallel (trafilatura extracts the main text), splits them into passages, and keeps the two per page that best match the question:

- Score = query words present + a small bonus for numbers (facts usually carry numbers).
- For "latest/most recent/current" questions, passages containing the newest year on the page get a large bonus, which finds the last rows of long tables.
- The result ends with an instruction to answer only from these results, name the source, and say so if the answer is not there.

Result: "Who won the most recent Super Bowl?" went from a wrong 2021 answer to "Super Bowl LX, February 8 2026, Seattle 29 New England 13, per ESPN."

6.4 Max's own browser (browser.py, tools/browser.py)

Max drives the installed Chrome through **Playwright**, with a separate profile in [data/browser-profile](#). Chrome does not allow automation of the everyday profile, and a separate one keeps Max's logins apart.

- Launched **without** Chrome's automation flags; otherwise Google shows a bot check.
- Launched with autoplay allowed, so videos Max opens actually play (a fresh profile would open YouTube paused).

Finding what to click

A small script runs in the page and tags every visible clickable or typeable element with a number ([data-max-id](#)), recording its text, role, position, whether it is in the main content (not the header, navigation or footer), whether it contains an image, and its link. Python then picks the element:

- By name: fuzzy match of the words you said against element text; clear winners are clicked directly.
- By position ("the second video"): main-content links with a headline or thumbnail, in reading order. On a page that is mostly videos, only video links count, so a channel card is skipped. A thumbnail is swapped for the title link on the same row (same destination, readable name).
- Ambiguous: a numbered list goes back to the model to choose from.
- Pages that draw themselves late (YouTube) are given a moment and rescanned before giving up.

Browser tools

Tool	Purpose
browser_click	Click by name, position or number
browser_type	Type into a field; search boxes submit automatically
browser_navigate	Scroll, top/bottom, back, forward, reload
browser_media	Play, pause, mute, unmute, restart the page's video (also inside embedded players)
browser_read	Read the page to answer a question or summarize it
browser_tabs	List, switch, close, new tab

Browser safety

Clicks on buttons whose text looks like buy, pay, order, send, post, subscribe, delete, confirm and similar ask for a spoken yes. Pressing Enter anywhere except a search box also asks. Password fields are never typed into.

7. Memory and reminders

7.1 The memory store (memory.py)

One SQLite file, `data/max.db`, holds four kinds of data: facts you ask Max to remember, every conversation turn (with the tools used), an action log for the dashboard, and reminders. Facts and turns also get an embedding from **bge-small-en** (fastembed, ONNX on the CPU, about 28 ms), so they can be found by meaning. The vectors are kept in memory and compared with a dot product, which is instant at personal scale; no vector database is needed.

- "Remember that my exam is on Friday" saves a fact; a near-identical fact is updated instead of duplicated.
- "What did I ask you yesterday?" searches the conversation log by date or meaning.
- "Forget ..." finds the closest fact and asks you to confirm before deleting.

- Everything survives restarts: it lives on disk, not in the program.

7.2 Reminders (reminders.py)

Times are parsed from natural language with dateparser plus a few rewrites it doesn't handle on its own ("next Monday", "tomorrow morning", "tonight at 9", "Thursday evening"). A date without a time means 9 AM. A background thread checks every 10 seconds; a due reminder shows a Windows notification immediately and is spoken as soon as Max is idle, so it never talks over a conversation. Reminders that came due while Max was off are announced at the next start ("While you were away...").

8. Canvas and the daily digest

8.1 Canvas through the calendar feed (canvas.py)

ASU does not allow students to create Canvas API tokens, so Max reads the read-only **calendar feed** (Canvas > Calendar > Calendar Feed). The link works like a password, so it lives in `secrets.yaml`, which git ignores.

- Items with UID `event-assignment-...` are assignments and quizzes; Canvas gives only a date, so they count as due by the end of that day.
- Items with UID `event-calendar-event-...` are class sessions, with Zoom links pulled from the description.
- Cross-listed courses repeat every session once per section ("2024 Spring", "2025 Fall C" ...); only the section matching the current term is kept. On the real feed: 123 entries became 19 assignments and 28 class sessions.
- The feed is cached for 15 minutes; if Canvas is down (it has maintenance windows), the last copy is used.

8.2 The daily digest (digest.py)

Every morning at 7:00, Max emails a summary: what's due today and in the next three days, today's classes with Zoom links, and today's reminders, opened by two sentences from the local model on what to focus on. It is sent through Gmail with an app password (in `secrets.yaml`). If the laptop was off at 7, it goes out when Max starts, until 6 PM; a key in the database makes sure it is never sent twice in a day. "What does my day look like?" gives a spoken version and "email me my summary" sends it on demand.

9. Meeting and lecture notes (notes.py)

"Hey Max, take notes" (or the record button) listens to an in-person **lecture** through the mic, or to a **meeting** on the laptop by capturing the laptop's audio output (WASAPI loopback, via the soundcard library) mixed with the mic. When it's over, Max writes structured notes.

1. **Chunking:** audio is cut into 25-32 second chunks at the quietest moment near the end, so words aren't split.

2. **Live captions:** a small streaming model (sherpa-onnx zipformer "Kroko", CPU) turns the audio into words about half a second after they are said, with punctuation and capitals. They appear in the dashboard, the overlay pill and the phone app.
3. **Accurate transcription:** each chunk also goes to Whisper while the recording continues, and Whisper's text replaces the live captions for that stretch (roughly every 30 seconds). Only Whisper's text is saved. The prompt is the session title plus the last sentences heard, not the voice-command vocabulary, which keeps technical terms consistent.
4. **Section notes:** every ~700 words, the model writes bullet notes for that part, in the background.
5. **Final notes:** at the end the section notes are merged into Summary, Key points, Details, Action items & deadlines, and Open questions. This two-level approach is needed because the 4b model reads about 4k tokens at a time and an hour-long lecture is about 13k.
6. **Exact-quote deadlines:** sentences from the transcript that mention due dates, quizzes, exams and so on are appended word for word.

9.1 Live captions

Whisper reads a whole chunk at once, so on its own text appeared 30-40 seconds after it was said. Re-running Whisper every second on the last few seconds would have cost far more CPU than the laptop can spare next to the wake word. Instead a streaming model built for word-by-word output runs alongside it. Two candidates were benchmarked on a 136-second lecture:

Model	Compute for 136 s of audio	Output
Kroko streaming zipformer (chosen, 70 MB)	3.9 s (about 3% of real time)	Punctuation and capitals; got "RDF", "SQL", "knowledge graphs" right
NeMo FastConformer 480 ms, int8 (106 MB)	9.9 s (about 7%)	Lower case, no punctuation

The live text is rougher ("SPARQL" came out as "sparkle"), which is why it is only a preview: a live line is dropped as soon as most of it is covered by a Whisper chunk, so the same words are never shown twice at a chunk boundary. Decoding runs in its own thread so audio capture never waits on it, and if the model is missing or fails, recording carries on without captions.

9.2 Study summaries

Next to Notes and Transcript, a **Summary** button asks the local model for a short study version: a TL;DR, three to six key takeaways and a to-do list. It is written on first request and cached as [summary.md](#) beside the notes. The exact deadline quotes are removed before the model sees the notes and appended afterwards unchanged, so a time can't be rewritten.

Why the exact quotes

In testing the model once turned "11:59 PM" into "5:59 PM" in its summary while the transcript was correct. Deadlines are too important to leave to paraphrase, so the verbatim section is built by code, not by the model.

Notes are saved as `notes.md` and `transcript.md` in `Documents\Max Notes\<date> <title>`. No audio is kept. On a 2-minute test lecture the notes captured every definition, the assignment (due Friday, October 9, 11:59 PM, as a PDF on Canvas), the Monday quiz and the moved office hours; the write-up took about a minute after stopping.

10. Dashboard, API and desktop overlay

10.1 Event bus and API (`events.py`, `server.py`)

Everything Max does is published once on an event bus: stage changes, what it heard, tool calls, answers, the mic level, reminders and notes. A FastAPI server in a background thread, bound to `127.0.0.1:8765`, streams those events over a WebSocket and offers a REST API for the conversation log, facts, reminders, notes, typed commands and approvals. It is the same API the phone app uses (and the watch app will), reached through Tailscale.

- **Approvals race:** a risky action opens an approval that can be answered by voice or by a click in the dashboard; whichever comes first wins, and a click stops the listening early.
- **Shared controls:** pause, quit and "listen now" (push-to-talk) are one object used by the tray icon, the overlay and the API.
- A lock makes sure a typed and a spoken command never run at the same time.

10.2 The dashboard (`dashboard/`)

A React + Vite + TypeScript app with a "mission control" look (deep navy, cyan glow, a grid, HUD corners, monospace labels). The centrepiece is a canvas **orb** driven by Max's state and the live mic level: it breathes when idle, ripples to your voice, swirls when thinking, pulses when speaking and turns amber when an approval is waiting. Around it: service status, the conversation with the tools used for each answer (and a box to type commands), approvals, a live activity feed, and a drawer for memory, reminders and notes (rendered Markdown, with the transcript one click away). Fonts are bundled, so nothing loads from the internet.

10.3 Always on: tray, auto-start and overlay

- **Background mode** (`--tray`): no console window; a tray icon offers Open dashboard, Pause listening, Sleep models (free the GPU) and Quit. Only one copy of Max can run.
- **Auto-start** (`install_autostart.ps1`): a scheduled task starts Max 30 s after login and restarts it after a crash; a clean stop is respected.

- **Desktop overlay** ([max_assistant/overlay](#)): an always-on-top pill (pywebview on Windows' built-in Edge WebView) running as its own process, so it keeps working when Max is stopped and can start it. Buttons: push-to-talk, record notes, chats, memories, due today, notes, and a two-step start/stop.

11. The phone app and remote access (Phase 4)

A native Android app ([android/](#), Kotlin and Jetpack Compose) gives Max a second front end. Nothing about the brain moved: transcription, the agent and the models still run on the laptop, and the phone talks to the same API and event stream as the dashboard.

11.1 Reaching the laptop from anywhere

- **Tailscale** puts the laptop and the phone on a private, encrypted network. `tailscale serve --bg 8765` publishes the API at <https://<laptop>.<tailnet>.ts.net> inside that network only, with a real HTTPS certificate. The API itself stays bound to 127.0.0.1.
- **Token (remote.py)**: a random token is created once in [data/phone_token.txt](#) (never committed). A request counts as local only if it comes from 127.0.0.1 **without** Tailscale's forwarding headers; anything else must present the token (a header, or a query parameter for the WebSocket) or gets a 401. The token is compared in constant time.
- **Pairing**: the dashboard's Phone tab shows a QR code ([max://pair?url=...&token=...](#)); the app scans it. The endpoint that reveals the token answers only on the laptop itself.

11.2 What the app does

- **Tap to talk**: the phone records 16 kHz audio and stops on its own after a pause. The laptop transcribes it with Whisper, runs the agent and returns the reply, which the phone plays in Max's own Piper voice ([/api/voice](#), [/api/speak](#)).
- **Screens**: the Max screen (centrepiece, last question, Max's answer with times and days picked out, a "Tap to talk" pill and a keyboard button), conversation, today (Canvas items, classes, reminders you can add or cancel), memory (facts, meeting/lecture notes with the live view and summaries) and settings.
- **Always connected**: a foreground service keeps one WebSocket to the event stream, reconnects with backoff when the laptop sleeps or the network changes, and restarts after a reboot. Events become notifications: reminders, notes ready and approvals.
- **Listening sheet**: a see-through panel that slides up over any app and starts listening at once. It opens from the side key (Max registers as an assistant app), a Quick Settings tile, a home-screen widget or the app icon's shortcut, listens again if Max asks a question, and closes itself after the reply. Because it is a visible screen, the phone's "microphone only while using the app" permission is enough; an always-listening wake word on the phone was rejected for battery reasons, as on the watch.

11.3 Approvals from the phone

Every command is tagged with where it came from (voice, dashboard or phone). A risky action started from the phone never asks the laptop's microphone, since nobody may be sitting there: it waits up to 90 seconds for an **Approve / Deny** tap, shown as a notification with buttons and inside the app. No answer means no. The laptop also stays quiet: its spoken "let me look that up" fillers are skipped for phone commands.

11.4 Appearance: colours, centrepieces and fonts

The look was chosen through rounds of mockups on a design canvas: four directions, then a premium dark set, then combinations, ending with reactor rings above the answer set in a large serif. Rather than fixing one theme, the app now has **Settings → Appearance**, applied live with a preview and saved on the phone:

Setting	Choices
Colour	Gold (default), Rose, Ice, Emerald, Violet, Platinum. Each sets the accent, background glow, text and core-glow colours together.
Centrepiece	Reactor (turning arcs and a breathing core that swells with your voice), Waveform (voice bars), or Words (just the answer).
Font	Seven pairings of a display face and a body face: Classic (Instrument Serif + Manrope), Elegant, Editorial, Modern, Geometric, Tech, Soft. Bundled, so they work offline (SIL Open Font Licence).
Options	Answer size S/M/L, glow Subtle/Normal/Strong, animation on/off, voice bars around the ring (reactor), thin or bold bars (waveform).

The colours and fonts are read from one composition local, so every screen, the listening sheet and the menu bar follow the choice without each screen knowing about themes. The centrepieces are drawn with the Compose canvas from Max's state (idle, listening, thinking, speaking) and the live microphone level.

- **Settings rebuilt as rows:** on a phone, two or three buttons side by side (Listen now / Pause mic / Resume, Add tile / Add widget) squeezed and wrapped their labels. Settings is now full-width rows grouped into sections, with three equal tiles for the laptop controls.
- **Checking layouts without a phone:** Paparazzi renders the main screens to images on the laptop (layoutlib, no device or emulator), which is how the settings layout and the fonts were checked. Variable-weight font files rendered at the wrong weight there, so the app bundles one file per weight.

11.5 Pages open on the phone

Asked on the phone to "search for some cute dog photos and show me", Max replied "I'll open a search..." and nothing happened. The log showed two faults: the model called the tab tool of its laptop Chrome window, which failed because no page was open, and then promised an action it never took. Even when it worked, a search would have opened on the laptop, out of sight.

- Commands from the phone carry a short note telling the model so: websites, searches and videos go through `open_website`, phone apps through the phone tools, laptop apps through `open_app`.
- From the phone, `open_website` publishes a `phone_open` event instead of driving the laptop's Chrome; the app opens the link straight away when it is on screen, or posts a tap-to-open notification. The laptop-browser tools answer a phone command with a pointer to `open_website` instead of running.
- The false-claim check now also catches "I'll open / search / show..." when nothing was opened, and a request for photos, images or a website that called no opening tool.

11.6 Max acts on the phone

Max can now do things on the phone, whether the request comes from the phone or by voice to the laptop ("set a 10 minute timer on my phone"). The laptop publishes a `phone_action` event; the app carries it out with Android intents and posts the outcome back (`phone.py, /api/phone/result`), so Max can say "Alarm set for 6:30 AM" or ask "Which one: Alex Kim, Alex Ruiz?". When the link connects, the phone also posts its installed apps and battery, so app names are matched on the laptop.

Tool	What happens on the phone
<code>phone_open_app</code>	Opens any installed app by name
<code>phone_call</code>	Calls a contact or number. Risky: Max asks first
<code>phone_message</code>	SMS or WhatsApp to a contact, opened with the text typed in; the user taps Send
<code>phone_alarm / phone_timer</code>	Set straight away (no clock app screen)
<code>phone_navigate</code>	Google Maps directions
<code>phone_play</code>	Spotify or YouTube Music play-from-search; YouTube search
<code>phone_calendar_add / phone_email</code>	Opened filled in; the user taps Save / Send
<code>phone_notifications</code>	Recent notifications, for "what did I miss?"
<code>phone_status</code>	Battery, charging, network

- **Permissions, each off until granted** in Settings → Phone powers: contacts (so "Mom" resolves to a number), calls (otherwise Max dials and the user taps Call), notification access (notifications are kept in memory on the phone, never saved) and "display over other apps".
- **Android only lets an app start another app while the user is looking at it.** Requests from the phone happen at once; a request from the laptop with the phone in a pocket becomes a tap-to-open notification, unless "display over other apps" is allowed.
- **Tested with the real model** on nine everyday requests (timer, alarm, open Instagram, call Mom, WhatsApp Alex, what did I miss, battery, directions, play lofi): each reached the right action, and the call stopped at "Call Mom from your phone?". One bug surfaced: the model asked for notifications from the app "all", which would have matched the Gallery app; it is now treated as every app.

12. Reliability on Windows

12.1 The "only listens when its window is in front" bug

Max stopped responding whenever another window was active, first with VS Code and then even in its own Command Prompt window. The heartbeat log gave it away: the window title read **"Select Max - run.bat"**. Clicking into a console puts it into text-selection mode, and while a console is not being read, Windows blocks any program the moment it prints. Max heard "Hey Max", tried to print "Listening...", and froze; the audio kept piling up in the queue. VS Code's terminal does the same when it is not the active window.

Fix ([winutil.py](#)):

- All console output goes through a **background writer thread**; if the console stalls, only that thread waits.
- Click-to-select (QuickEdit) is turned off in Max's console.
- Windows power throttling ("efficiency mode") is disabled for Max's process and its priority raised slightly.

A test reproduces the freeze with a pipe nobody reads: plain printing froze, the background writer kept running.

12.2 Heartbeat and stack dumps ([diagnostics.py](#))

Every 30 seconds Max logs one line: what the main loop is doing and for how long, how many microphone frames arrived (counted on the audio thread, independent of the main loop), the input level, how much audio is queued, and which window is in front. If the loop sits in one busy stage for over 60 seconds, the stack of every thread is written to [logs/stacks.log](#), showing exactly where it is stuck.

12.3 Launching

`start_max.bat` starts Max in its own minimized window. `run.bat --text` gives a typed mode for testing without a microphone; `--wake-test` shows live wake word detections for tuning.

13. Configuration reference (config.yaml)

Section	Key settings
assistant	name (Max), user_name, max_steps (8)
llm	host, fast_model, planner_model, keep_alive, temperature, think, num_ctx, num_predict, fast_model_gpu_layers
wake_word	engine (sherpa / openwakeword), phrase, cooldown_s; sherpa: model_dir, threshold, boost, streams, max_gain
web	searxng_url, fallback_duckduckgo, results, timeout_s, read_pages
browser	enabled, profile_dir, channel (chrome / chromium), headless
audio	input/output device, silence_s, early_s, max_silence_s, no_speech_timeout_s, max_record_s, vad_sensitivity, chime, follow_up, follow_up_timeout_s
stt	model, device, compute_type, prompt, fast_model_dir (off), fast_min_snr_db
tts	engine, piper_voice, speed
safety	confirm_risky, confirm_timeout_s
memory	enabled, db (data/max.db)
digest	enabled, time (07:00), weekends, catch_up_until (18:00)
notes	enabled, folder (~/Documents/Max Notes), file (quick notes), include_mic_in_meetings, live_captions, live_model
dashboard	enabled, host (127.0.0.1), port (8765), speak_typed_replies
phone	enabled, public_url (blank = auto-detect the Tailscale name), token_file, approval_timeout_s (90)
secrets.yaml	Not committed: canvas.ics_url, email.smtp_user / app_password / to. The phone token lives in data/ (also not committed).

14. Project layout

Path	Contents
max_assistant/main.py	Entry point, voice loop, text mode, follow-up conversation, fillers
max_assistant/agent.py	Tool-calling loop, escalation, safety gate, history
max_assistant/llm.py	Ollama client (tools, GPU layer forcing, preload/unload)
max_assistant/audio.py / listen.py	Microphone, recording, early transcription, end of turn
max_assistant/wakeword.py	sherpa keyword spotting (staggered streams, auto gain), openWakeWord fallback
max_assistant/stt.py / tts.py	Whisper (+ optional Moonshine), Piper voice
max_assistant/web.py / browser.py	Search + page reading; Max's Chrome window
max_assistant/winutil.py / diagnostics.py	Console unblocking, throttling opt-out, heartbeat
max_assistant/tools/	registry.py, system.py, web.py, browser.py
max_assistant/memory.py / reminders.py	Facts, conversation log, actions, reminders; scheduler and notifications
max_assistant/canvas.py / digest.py	Canvas calendar feed; daily digest email
max_assistant/notes.py / live.py	Meeting / lecture capture, transcription, notes and summaries; live captions
max_assistant/events.py / server.py	Event bus, approvals, controls, command origin; FastAPI + WebSocket
max_assistant/remote.py	Phone access: token, local-vs-remote check, audio in/out, pairing QR code
max_assistant/phone.py, tools/phone.py	Phone actions: the request/answer link to the app and the 11 phone tools
max_assistant/overlay/, tray.py	Desktop overlay; tray icon
dashboard/	React + Vite + TypeScript dashboard
android/	Kotlin + Jetpack Compose phone app (Look.kt themes, Centrepiece.kt, PhoneActions.kt, the notification listener); build.ps1 builds and installs it (JDK and SDK in C:\max-android, no Android Studio)

Path	Contents
tests/test_max.py	102 tests; no microphone, GPU, Ollama or internet needed (plus Paparazzi screen renders in android/)
config.yaml, setup.ps1, run.bat, start_max.bat, overlay.bat, install_autostart.ps1	Settings, setup, launchers, auto-start
docker-compose.yml, searxng/	Private search engine

15. How it is tested

The 102 automated tests run anywhere: the language model, microphone, browser page, search engine, Canvas feed and embedding model are replaced by small fakes. They cover the tool registry, the agent loop (direct replies, escalation, history, false-claim nudges, direct routes), the wake-word logic, element picking, the listener's end-of-turn logic, speech-to-text routing, the non-blocking console, follow-up conversations, memory and reminders, Canvas parsing, the digest schedule, the API (approvals, controls, notes, summaries), phone access (token required remotely and through the Tailscale proxy, the token never handed out remotely, commands tagged by origin, tap-only approvals that time out to no, the voice endpoint, searches from the phone opening on the phone, the laptop-browser tools redirected, the reported "I'll open a search" failure replayed, phone actions and their answers, an offline or slow phone, and the full action round trip over the API), live captions (with a fake streaming recognizer, plus a real-model check that is skipped when the model is not downloaded) and the notes pipeline (chunking at pauses, section notes, exact-quote deadlines, Whisper replacing captions). One test rejects duplicate keys in config.yaml.

Behaviour that depends on real models was checked with scripted experiments instead of unit tests: synthetic voices through the real wake word detector at many volumes and timings, real Whisper and Moonshine on quiet and noisy clips, and scripted conversations against the real model driving a real (off-screen) Chrome window.

16. Key lessons

- **Measure the right quantity.** Token counts per reply exposed the reasoning model; frame counters and the front-window title exposed the console freeze.
- **Synthetic tests can mislead.** Moonshine matched Whisper on generated voices and failed on a real one.
- **Small models copy patterns.** History without tool calls taught the model to claim actions it never took.
- **Ground answers in sources.** Without reading pages, the 4b model invented answers and citations.

- **Never let an always-on loop wait on something it does not control:** console output, a slow page, a busy mic.
- **Don't let a small model paraphrase critical facts.** Deadlines are quoted verbatim from the transcript; memory and notes commands that the model mishandled go straight to their tools.
- **Benchmark before choosing a model.** Two streaming models looked similar on paper; on a real lecture one was twice as fast and added punctuation.
- **Configuration can fail silently.** A second `notes:` section in `config.yaml` quietly replaced the first, so meeting-notes settings were ignored for days. YAML keeps the last duplicate without complaint; a test now refuses duplicates.
- **Know where the user is.** The same words mean different things on the phone and at the laptop; telling the model where a command came from fixed "open YouTube" opening Spotify on the laptop.
- **Check designs where they will be used.** Buttons that fit a desktop mockup wrapped on a phone; offscreen renders of the real screens caught layout and font problems before the user did.
- **Don't trust where a request comes from just because it's local.** Through Tailscale, every phone request arrives from 127.0.0.1; the forwarding headers are what tell them apart.
- **Keep secrets out of the repo and out of screenshots.** Feed links and passwords live in a git-ignored file; UI checks use headless renders, not captures of the real screen.

17. What comes next

Phase 5 adds a Galaxy Watch companion (Wear OS, tap-to-talk) using the same API and pairing as the phone. **Phase 6** connects Gmail, Google Calendar and Tasks, followed by polish and a demo video. Now that the phone reaches Max from anywhere, the remaining limit is that the laptop has to stay on; moving the brain to an always-on box (a Mac mini or a small PC with a GPU) would only change the address in the app.